

Week 1 – JavaScript Basics: Detailed Exercises (Days 1-7)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 17 Aug 2026 – 23 Aug 2026 (Days 1-7)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm & 9:00-10:00pm** · Sat-Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Learning path: Read `00-LEARNING-PATH-COMPLETE.pdf` first — Phase 1.

Roadmap: `ROADMAP.md` · Theory: `01-javascript-complete.md`

Parallel (Day 8+): `08b-dsa-datastructures-fundamentals.md` §1

Interview Q&A: Tier 1 only (`17-interview-qa-80lpa-master.md` §1-2) — 2 Q&A/day from Day 22

How to read solutions in this PDF: Every **Complete Solution** is followed by a **Line-by-line explanation** table. Each code line has a `// comment` — the PDF expands these into a table so you understand every line, not just copy-paste.

Beginner rule: We do **not** use unexplained short forms. See `00-beginner-glossary.pdf` for every term. Examples: **Temporal Dead Zone** (not "TDZ"), **Read-Eval-Print Loop** (not "REPL"), **JavaScript** (not "JS"), **LeetCode problem 1** (not "LC 1").

Prerequisite chain

None → 01-javascript sections 1-5 (this week) → sections 6-11 (Week 2) → TypeScript (W)
Do NOT read yet: distributed systems, system design, advanced data structures and algo

Day 1 – Variables, Types, Coercion

Exercise 1.1: Variable Quiz (Hoisting & Coercion)

Problem statement

Predict the output of every `console.log` below, write predictions on paper, then run in Node.js and compare:

```
console.log(a); var a = 1;
console.log(b); let b = 2;
console.log(typeof null);
console.log([] + []);
console.log([] + {});
console.log({} + []);
```

Learning objective

Understand hoisting (`var` vs `let`), falsy values, and JavaScript type coercion rules for `+` with arrays and objects.

Prerequisites

- Theory: [01-javascript-complete.md §1 Variables & Data Types](#)

Step-by-step approach

- For each line, write what you think prints **before** running code.
- Identify which declarations are hoisted and initialized to `undefined`.
- Recall that `typeof null === "object"` (historical bug).
- Evaluate `[] + []` as string concatenation (`"" + ""`).
- Evaluate `[] + {}` and `{ } + []` — note `{ }` can be parsed as block vs object depending on context.
- Run in Node (`node quiz.js`) and annotate differences.
- Rewrite each surprising line in a comment explaining the actual behavior.

Hints

- `var` declarations hoist the name but not the assignment; `let` / `const` are in the **Temporal Dead Zone** (cannot use the variable until its declaration line runs).
- When `+` has an object operand, JavaScript calls **ToPrimitive** (convert to a simple value) / `toString()` — arrays become comma-joined strings.

3. At the start of a statement, `{` may start a **block** (group of statements), not an object literal — that changes `{}` + `[]`.

Exact Answer

Line	Output	Explanation
<code>console.log(a); var a = 1;</code>	<code>undefined</code>	<code>var a</code> is hoisted and initialized to <code>undefined</code> before assignment runs
<code>console.log(b); let b = 2;</code>	ReferenceError	<code>b</code> is in the Temporal Dead Zone until <code>let b = 2</code> executes
<code>console.log(typeof null)</code>	<code>"object"</code>	Historical JavaScript bug – <code>null</code> is not an object but <code>typeof</code> returns <code>"object"</code>
<code>console.log([] + [])</code>	<code>""</code>	Both arrays call <code>toString()</code> → <code>"" + ""</code> → empty string
<code>console.log([] + {})</code>	<code>"[object Object]"</code>	<code>[]</code> → <code>""</code> , <code>{}</code> → <code>"[object Object]"</code> via <code>toString()</code>
<code>console.log({} + [])</code>	<code>"[object Object]"</code> (Node Read-Eval-Print Loop) or <code>0</code> (script file)	In interactive Node, <code>{}</code> is an object literal; in a script starting with <code>{</code> , it may parse as an empty block + unary <code>+[]</code> → <code>0</code>

Complete Solution

```
// Run in Node: node quiz.js – saves file and execute with: node quiz.js
console.log(a); var a = 1;           // Prints undefined – `var a` is hoisted and ini
// console.log(b); let b = 2;       // Would throw ReferenceError – `b` is in Tempor
console.log(typeof null);           // Prints "object" – historical JavaScript bug;
console.log([] + []);               // Prints "" – both arrays call toString() → ""
console.log([] + {});               // Prints "[object Object]" – [] → "", {} → "[ob
console.log({} + []);               // Prints "[object Object]" in Read-Eval-Print L
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(a); var a = 1;</code>	Prints undefined – <code>var a</code> is hoisted and initialized to undefined before this line runs
<code>console.log(typeof null);</code>	Prints "object" – historical JavaScript bug; null is not actually an object
<code>console.log([] + []);</code>	Prints "" – both arrays call toString() → "" + "" → empty string
<code>console.log([] + {});</code>	Prints "[object Object]" – [] → "", {} → "[object Object]", then string concat
<code>console.log({} + []);</code>	Prints "[object Object]" in Read-Eval-Print Loop; in script files may parse as block + +[] → 0

Expected output / acceptance criteria

- Written predictions exist for all 7 lines before execution.
- You can explain every mismatch without looking at notes.
- Bonus: document outputs in `week-01/day-01-quiz.md`.

Where to practice

- Mozilla Developer Network — coercion: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition>
- Node.js Read-Eval-Print Loop: <https://nodejs.org/en/learn/command-line/run-nodejs-scripts-from-the-command-line>
- Local: `mern-mastery/week-01/day-01/quiz.js`

Common mistakes

- Assuming `typeof null` is "null".
- Forgetting **Temporal Dead Zone** error on `console.log(b)` before `let b`.
- Treating `[] + {}` the same in Read-Eval-Print Loop vs script file first line.

80 LPA Interview Twist

At Google/Meta, expect follow-ups on ECMAScript specification sections (**ToPrimitive**, **OrdinaryToPrimitive**) and a live ask to fix a production bug caused by `{ } + [ ]` parsing differently in bundlers vs Node — they want specification-level reasoning, not memorized outputs.

Interview connection

"Explain hoisting" and "what is `typeof null`?" appear in almost every JavaScript screen. Coercion questions test whether you read specifications or only write React.

Time estimate: 45 minutes

Exercise 1.2: FizzBuzz

Problem statement

Print numbers 1 through 100. Multiples of 3 → "Fizz", multiples of 5 → "Buzz", multiples of both → "FizzBuzz", otherwise the number.

Learning objective

Practice loops, modulo operator, and ordering conditional checks correctly.

Prerequisites

- Theory: [01-javascript-complete.md §1](#)

Step-by-step approach

1. Create `fizzbuzz.js` with a function `fizzBuzz(n = 100)`.
2. Loop `i` from 1 to `n` inclusive.
3. Check `% 15 === 0` first (both 3 and 5), then `% 3`, then `% 5`.
4. `console.log` the result for each `i`.
5. Extract logic into `getFizzBuzzLabel(i)` for testability.
6. Verify edge cases: 1, 3, 5, 15, 100.
7. Push to `mern-mastery/week-01`.

Hints

1. Check "FizzBuzz" (15) before Fizz or Buzz alone.
2. `%` gives remainder; `i % 3 === 0` means divisible by 3.
3. Return strings from a helper so you can unit-test without console output.

Exact Answer

- Output: 100 lines — 1, 2, Fizz, 4, Buzz, Fizz, 7, ... FizzBuzz at 15, 30, 45, 60, 75, 90, and 100 at the end.
- Key lines: 3→Fizz, 5→Buzz, 15→FizzBuzz (not Fizz then Buzz).

Complete Solution

```
function getFizzBuzzLabel(i) {      // Helper: returns the correct label for a si
  if (i % 15 === 0) return "FizzBuzz"; // Divisible by both 3 and 5 – check this F
  if (i % 3 === 0) return "Fizz";    // Divisible by 3 only
  if (i % 5 === 0) return "Buzz";    // Divisible by 5 only
  return String(i);                 // Not divisible by 3 or 5 – return the numbe
}

function fizzBuzz(n = 100) {        // Main function; default prints 1 through 10
  for (let i = 1; i <= n; i++) {    // Loop from 1 to n inclusive (not 0)
    console.log(getFizzBuzzLabel(i)); // Print one result per line
  }
}

fizzBuzz();                          // Run with default n = 100
```

Line-by-line explanation (javascript)

Line	What it does
<code>function getFizzBuzzLabel(i) {</code>	Helper: returns the correct label for a single number
<code> if (i % 15 === 0) return "FizzBuzz";</code>	Divisible by both 3 and 5 – check this FIRST
<code> if (i % 3 === 0) return "Fizz";</code>	Divisible by 3 only
<code> if (i % 5 === 0) return "Buzz";</code>	Divisible by 5 only
<code> return String(i);</code>	Not divisible by 3 or 5 – return the number as a string
<code>function fizzBuzz(n = 100) {</code>	Main function; default prints 1 through 100
<code> for (let i = 1; i <= n; i++) {</code>	Loop from 1 to n inclusive (not 0)
<code> console.log(getFizzBuzzLabel(i));</code>	Print one result per line
<code> fizzBuzz();</code>	Run with default n = 100

Expected output / acceptance criteria

- Lines 1–100 print correctly; 15 and 30 show `FizzBuzz` .
- Function is reusable with custom `n` .
- No duplicated loop logic.

Where to practice

- LeetCode FizzBuzz (412): <https://leetcode.com/problems/fizz-buzz/>
- Local: `node week-01/day-01/fizzbuzz.js`

Common mistakes

- Checking `% 3` and `% 5` in separate `if` blocks without `else if`, printing multiple labels.
- Off-by-one loops (0–99 instead of 1–100).

80 LPA Interview Twist

Senior rounds ask you to implement FizzBuzz as an extensible rules engine (e.g., add "Jazz" for multiples of 7) using Strategy pattern or a config array — testing whether you think beyond hard-coded if-else.

Interview connection

Classic warm-up; interviewers watch code structure and whether you test edge cases unprompted.

Time estimate: 30 minutes

Day 2-3 – Functions, Scope, Closures

Exercise 2.1: Closure Counter

Problem statement

Implement `createCounter(initial = 0)` returning an object with:

- `increment()` — add 1
- `decrement()` — subtract 1
- `reset()` — back to initial value
- `getValue()` — current count

Each call to `createCounter` must have **independent** state.

Learning objective

Use closures to encapsulate private state without classes.

Prerequisites

- Theory: [01-javascript-complete.md §2 Functions & Closures](#)

Step-by-step approach

1. Declare `createCounter(initial)`; store `let count = initial` in outer scope.
2. Return object with four methods closing over `count` and `initial`.
3. `increment / decrement` mutate `count`; `reset` sets `count = initial`.
4. `getValue` returns `count` without mutating.

5. Test two counters: `const a = createCounter(0); const b = createCounter(10);` — verify independence.
6. Avoid exposing `count` on the returned object.
7. Optional: add `incrementBy(n)`.

Hints

1. Variables inside `createCounter` are private to returned methods.
2. `reset` needs the original `initial`, not a changing baseline.
3. Arrow functions work for methods if you don't need `this`.

Exact Answer

- `c.increment(); c.increment()` → `getValue()` returns `7`
- `c.reset()` → `getValue()` returns `5` (original initial, not 0)
- Two instances `a` and `b` maintain independent counts

Complete Solution

```
function createCounter(initial = 0) { // Factory function; initial defaults to 0
  let count = initial;               // Private variable – only inner functions

  return {                           // Return an object with four methods (closure)
    increment() { count += 1; },      // Add 1 to private count
    decrement() { count -= 1; },      // Subtract 1 from private count
    reset() { count = initial; },     // Restore count to original initial value (not 0)
    getValue() { return count; },     // Read current count without changing it
  };
}

const c = createCounter(5);           // New counter starting at 5
c.increment();                       // count becomes 6
c.increment();                       // count becomes 7
console.log(c.getValue());            // Prints 7
c.reset();                           // count back to 5
console.log(c.getValue());            // Prints 5

const a = createCounter(0);          // Separate counter – independent closure
const b = createCounter(10);         // Another independent counter
a.increment();                       // Only a changes
console.log(a.getValue(), b.getValue()); // Prints 1, 10 – proves separate state
```

Line-by-line explanation (javascript)

Line	What it does
<code>function createCounter(initial = 0) {</code>	Factory function; initial defaults to 0 if not passed
<code>let count = initial;</code>	Private variable – only inner functions can access this
<code>return {</code>	Return an object with four methods (closure keeps count alive)
<code>increment() { count += 1; },</code>	Add 1 to private count
<code>decrement() { count -= 1; },</code>	Subtract 1 from private count
<code>reset() { count = initial; },</code>	Restore count to original initial value (not hard-coded 0)
<code>getValue() { return count; },</code>	Read current count without changing it
<code>const c = createCounter(5);</code>	New counter starting at 5
<code>c.increment();</code>	count becomes 6
<code>c.increment();</code>	count becomes 7
<code>console.log(c.getValue());</code>	Prints 7
<code>c.reset();</code>	count back to 5
<code>console.log(c.getValue());</code>	Prints 5
<code>const a = createCounter(0);</code>	Separate counter – independent closure
<code>const b = createCounter(10);</code>	Another independent counter
<code>a.increment();</code>	Only a changes
<code>console.log(a.getValue(), b.getValue());</code>	Prints 1, 10 – proves separate state

Expected output / acceptance criteria

```
const c = createCounter(5);
c.increment(); c.increment(); // getValue() === 7
c.reset(); // getValue() === 5
```

Two instances never share state.

Where to practice

- Local: `mern-mastery/week-01/day-03/counter.js`

- MDN Closures: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures>

Common mistakes

- Storing count on `this` without proper binding.
- Using a global variable shared across instances.
- Resetting to 0 instead of `initial`.

80 LPA Interview Twist

Meta/Google ask for thread-safe or async-safe counters, or a `createCounter` that supports max/min bounds and emits events — testing closure design under concurrency and **Application Programming Interface** extensibility.

Interview connection

"Implement once/ debounce / memoize" all rely on closures. Counter is the minimal pattern.

Time estimate: 45 minutes

Day 4 – The `this` Keyword

Exercise 4.1: `this` Prediction

Problem statement

Predict outputs, then verify:

```
const obj = {
  name: "Obj",
  greet: function() { return this.name; },
  greetArrow: () => this?.name,
};
console.log(obj.greet());
console.log(obj.greet.call({ name: "Other" }));
console.log(obj.greetArrow());
```

Learning objective

Apply the four `this` rules: default, implicit, explicit (`call` / `apply` / `bind`), and `new`.

Prerequisites

- Theory: [01-javascript-complete.md §3 this keyword](#)

Step-by-step approach

1. Write predicted outputs for all three logs.
2. For `greet()`, identify implicit binding → `this === obj`.
3. For `.call({ name: "Other" })`, explicit binding overrides.
4. For `greetArrow`, recall arrows inherit `this` from enclosing lexical scope (often `undefined` in modules/strict).
5. Run in Node and document.
6. Add two more cases: detached `const fn = obj.greet; fn()` and `new (function() { ... })`.
7. Summarize rules in your own words in notes.

Hints

1. Regular methods get `this` from call site; arrows from definition site.
2. `call` first argument becomes `this`.
3. In strict mode, default `this` is `undefined`, not `global`.

Exact Answer

Call	Output	Rule
<code>obj.greet()</code>	<code>"Obj"</code>	Implicit binding – <code>this</code> is <code>obj</code>
<code>obj.greet.call({ name: "Other" })</code>	<code>"Other"</code>	Explicit binding via <code>call</code>
<code>obj.greetArrow()</code>	<code>undefined</code> (module/strict) or global name	Lexical <code>this</code> from enclosing scope, not <code>obj</code>

Complete Solution

```
"use strict"; // Strict mode – detached calls get undefined

const obj = { // Object literal with two method styles
  name: "Obj", // Property accessed via this.name in regular function
  greet: function () { return this.name; }, // Regular function – this set by HOW
  greetArrow: () => this?.name, // Arrow – this from OUTER scope (module to global)
};

console.log(obj.greet()); // "Obj" – implicit binding: this
console.log(obj.greet.call({ name: "Other" })); // "Other" – explicit binding via call
console.log(obj.greetArrow()); // undefined – arrow ignores obj, uses global

const fn = obj.greet; // Detach method from object – loses implicit binding
// console.log(fn()); // TypeError in strict mode – this is undefined

function Person(name) { // Constructor function (uses `new` binding)
  this.name = name; // `new` creates empty object and sets this to it
}

const p = new Person("New"); // new binding – this is the new instance
console.log(p.name); // "New" – property set on instance
```

Line-by-line explanation (javascript)

Line	What it does
<code>"use strict";</code>	Strict mode – detached calls get undefined this, not global
<code>const obj = {</code>	Object literal with two method styles
<code> name: "Obj",</code>	Property accessed via this.name in regular function
<code> greet: function () { return this.name;</code> <code>},</code>	Regular function – this set by HOW it's called
<code> greetArrow: () => this?.name,</code>	Arrow – this from OUTER scope (module top = undefined)
<code> console.log(obj.greet());</code>	"Obj" – implicit binding: this === obj
<code> console.log(obj.greet.call({ name:</code> <code> "Other" }));</code>	"Other" – explicit binding via call()
<code> console.log(obj.greetArrow());</code>	undefined – arrow ignores obj, uses lexical this
<code> const fn = obj.greet;</code>	Detach method from object – loses implicit binding
<code> function Person(name) {</code>	Constructor function (uses new binding)
<code> this.name = name;</code>	new creates empty object and sets this to it
<code> const p = new Person("New");</code>	new binding – this is the new instance
<code> console.log(p.name);</code>	"New" – property set on instance

Expected output / acceptance criteria

- All predictions written first; explanations match runtime.
- You can teach implicit vs explicit binding aloud in 60 seconds.

Where to practice

- Theory cross-ref: [01-javascript-complete.md §3](#)
- Local: `week-01/day-04/this-quiz.js`

Common mistakes

- Expecting arrow methods to use object as `this`.
- Confusing `call` argument order with `bind` partial application.

80 LPA Interview Twist

At L4/L5, expect "implement `Function.prototype.call`" or explain how React 19 / class fields

avoid `bind` — they probe whether you understand `this` at the spec level, not just prediction quizzes.

Interview connection

React class components, event handlers, and `bind` in legacy codebases — `this` bugs still appear in maintenance interviews.

Time estimate: 40 minutes

Day 5 – LeetCode: Two Sum (LeetCode problem 1)

Problem statement

Given an integer array `nums` and `target`, return indices `[i, j]` such that `nums[i] + nums[j] === target`. Exactly one solution exists; don't reuse the same element twice.

Learning objective

Use a **hash map** (key-value lookup table) for **linear time** lookup instead of **quadratic time** nested loops.

Prerequisites

- Theory: [08-dsa-patterns-complete.md section 3 Hash Map](#)
- Week 1 JavaScript: arrays, objects as maps

Step-by-step approach

1. Read problem and examples on LeetCode; note index return requirement.
2. Brute force: nested loops — implement first to verify understanding.
3. Optimize: one pass — for each `nums[i]`, check if `target - nums[i]` exists in map.
4. Store `{ value: index }` in `Map` or object as you iterate.
5. Return `[map.get(complement), i]` when complement found.
6. Handle edge case: negative numbers and duplicates.
7. Analyze **linear time** (one loop), **linear space** (map grows with input).

Hints

1. Store indices in the map, not just booleans.
2. Check complement **before** adding current index to map (avoids using same element twice).
3. `Map` avoids key coercion issues with `0` and `-0`.

Exact Answer

- `twoSum([2,7,11,15], 9) → [0, 1]`

- Time: **linear** (one pass), Space: **linear** (hash map size grows with input)

Complete Solution

```
function twoSum(nums, target) { // nums = array, target = sum we need
  const map = new Map(); // Map stores value → index for constant-time lookup

  for (let i = 0; i < nums.length; i++) { // One pass through the array (linear time)
    const complement = target - nums[i]; // What number do we need with nums[i]?
    if (map.has(complement)) { // Did we already see that complement?
      return [map.get(complement), i]; // Yes – return [earlier index, current index]
    }
    map.set(nums[i], i); // Store current value and its index for future lookups
  }
  return []; // No pair found (problem guarantees one solution)
}

console.log(twoSum([2, 7, 11, 15], 9)); // [0, 1] – nums[0]+nums[1] = 2+7 = 9
```

Line-by-line explanation (javascript)

Line	What it does
<code>function twoSum(nums, target) {</code>	nums = array, target = sum we need
<code>const map = new Map();</code>	Map stores value → index for constant-time lookup
<code>for (let i = 0; i < nums.length; i++) {</code>	One pass through the array (linear time)
<code>const complement = target - nums[i];</code>	What number do we need with nums[i]?
<code>if (map.has(complement)) {</code>	Did we already see that complement?
<code>return [map.get(complement), i];</code>	Yes – return [earlier index, current index]
<code>map.set(nums[i], i);</code>	Store current value and its index for future lookups
<code>return [];</code>	No pair found (problem guarantees one solution)
<code>console.log(twoSum([2, 7, 11, 15], 9));</code>	[0, 1] – nums[0]+nums[1] = 2+7 = 9

Expected output / acceptance criteria

- `twoSum([2,7,11,15], 9) → [0, 1]`
- Passes all LeetCode test cases.
- Can explain brute → optimal transition.

Where to practice

- <https://leetcode.com/problems/two-sum/>
- NeetCode video: <https://neetcode.io/problems/two-sum>

Common mistakes

- Returning values instead of indices.
- Adding current element to map before checking complement.
- Using same index twice when `target === 2 * nums[i]`.

80 LPA Interview Twist

Follow-up: return all pairs, or handle sorted array with two pointers in **constant extra space** — Google often extends Two Sum into 3Sum/4Sum and asks you to justify when hash map vs two pointers wins.

Interview connection

#1 most attempted LeetCode problem; pattern appears in 3Sum, subarray sum, and pair problems.

Time estimate: 45–60 minutes

Day 6 – Arrays & LeetCode 242, 121

Exercise 6.1: Array Manipulation

Problem statement

Given `[3, 1, 4, 1, 5, 9, 2, 6]`:

1. Remove duplicates without `Set`
2. Find the second largest unique value
3. Rotate array right by `k = 3` positions

Learning objective

Manipulate arrays with loops, indexing, and in-place techniques.

Prerequisites

- Theory: [01-javascript-complete.md §5 Arrays](#)

Step-by-step approach

1. **Dedupe**: iterate and push to new array only if not already included (`includes` or index check).
2. **Second largest**: dedupe first, sort descending, return index 1 — or one-pass track `largest` and `secondLargest`.

3. **Rotate:** slice `arr.length - k` to end + beginning, or reverse whole array three times trick.
4. Write tests for `k > length` (use `k % n`).
5. Document time/space for each approach.
6. Avoid mutating input unless specified.

Hints

1. For one-pass second largest, update both when you see a new max.
2. Rotate by 3 on length 8 → last 3 elements move to front.
3. `k % arr.length` handles k larger than array.

Exact Answer

- Dedupe → `[3, 1, 4, 5, 9, 2, 6]` (order preserved)
- Second largest → `6`
- Rotate by 3 → `[2, 6, 3, 1, 4, 1, 5, 9]`

Complete Solution

```
const arr = [3, 1, 4, 1, 5, 9, 2, 6]; // Input array for all three tasks

function dedupe(a) { // Remove duplicates without Set
  const result = []; // New array for unique values
  for (const x of a) { // Loop each element
    if (!result.includes(x)) result.push(x); // Push only if not already in result
  }
  return result;
}

function secondLargest(a) { // One-pass linear time – no sorting needed
  let largest = -Infinity, second = -Infinity; // Track top two values
  for (const x of a) {
    if (x > largest) { // New maximum found
      second = largest; // Old max becomes second
      largest = x; // Update max
    } else if (x > second && x < largest) { // Between second and largest
      second = x;
    }
  }
  return second;
}

function rotateRight(a, k) { // Rotate array right by k positions
  const n = a.length; // Array length
  if (n === 0) return []; // Edge case: empty array
  k = k % n; // Handle k > length (e.g. k=11 on length 8)
  return [...a.slice(n - k), ...a.slice(0, n - k)]; // Last k elements + first n-k
}

console.log(dedupe(arr)); // [3, 1, 4, 5, 9, 2, 6]
console.log(secondLargest(arr)); // 6
console.log(rotateRight(arr, 3)); // [2, 6, 3, 1, 4, 1, 5, 9]
```

Line-by-line explanation (javascript)

Line	What it does
<code>const arr = [3, 1, 4, 1, 5, 9, 2, 6];</code>	Input array for all three tasks
<code>function dedupe(a) {</code>	Remove duplicates without Set
<code>const result = [];</code>	New array for unique values
<code>for (const x of a) {</code>	Loop each element
<code>if (!result.includes(x)) result.push(x);</code>	Push only if not already in result
<code>function secondLargest(a) {</code>	One-pass linear time – no sorting needed
<code>let largest = -Infinity, second = -Infinity;</code>	Track top two values
<code>if (x > largest) {</code>	New maximum found
<code>second = largest;</code>	Old max becomes second
<code>largest = x;</code>	Update max
<code>} else if (x > second && x < largest) {</code>	Between second and largest
<code>function rotateRight(a, k) {</code>	Rotate array right by k positions
<code>const n = a.length;</code>	Array length
<code>if (n === 0) return [];</code>	Edge case: empty array
<code>k = k % n;</code>	Handle k > length (e.g. k=11 on length 8 → k=3)
<code>return [...a.slice(n - k), ...a.slice(0, n - k)];</code>	Last k elements + first n-k elements
<code>console.log(dedupe(arr));</code>	[3, 1, 4, 5, 9, 2, 6]
<code>console.log(secondLargest(arr));</code>	6
<code>console.log(rotateRight(arr, 3));</code>	[2, 6, 3, 1, 4, 1, 5, 9]

Expected output / acceptance criteria

- Dedupe → [3, 1, 4, 5, 9, 2, 6] (order preserved)
- Second largest → 6
- Rotate by 3 → [2, 6, 3, 1, 4, 1, 5, 9]

Where to practice

- Local: `week-01/day-06/arrays.js`

- MDN Array methods: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

Common mistakes

- Using `Set` when exercise forbids it.
- Off-by-one on rotate slice indices.
- Confusing second largest with second element after sort.

80 LPA Interview Twist

Amazon/Google ask for in-place rotation (**constant extra space** via triple-reverse) and dedupe in a streaming context where the array doesn't fit in memory — expect complexity and input/output tradeoff discussion.

Interview connection

Array rotation (LeetCode problem 189) and top-K elements are common follow-ups.

Time estimate: 60 minutes

Exercise 6.2: LeetCode problem 242 – Valid Anagram

Problem statement

Given strings `s` and `t`, return true if `t` is an anagram of `s`.

Learning objective

Character frequency counting with a fixed-size array or map.

Prerequisites

- Hash map pattern: [08-dsa-patterns-complete.md §3](#)

Step-by-step approach

1. If lengths differ, return false immediately.
2. Build frequency map for `s` (26-letter array for lowercase English).
3. Decrement counts traversing `t`; any negative or leftover → false.
4. Alternative: sort both strings and compare — discuss tradeoffs.
5. Submit on LeetCode; note **linear time** vs **sorting time** ($n \log n$).

Hints

1. `charCodeAt(i) - 97` indexes a-z in a 26-slot array (97 is character code of `'a'`).
2. Early exit on length mismatch saves work.
3. Unicode inputs need Map, not fixed array.

Exact Answer

- `isAnagram("anagram", "nagaram") → true`
- `isAnagram("rat", "car") → false`
- Time: **linear** (one pass), Space: **constant** (26-letter alphabet) or **linear in unique letters** with Map

Complete Solution

```
function isAnagram(s, t) { // Return true if t is rearrangement of s
  if (s.length !== t.length) return false; // Different lengths cannot be anagrams
  const count = new Array(26).fill(0); // Frequency bucket for a-z (26 letters)

  for (let i = 0; i < s.length; i++) { // Single pass – increment for s, decrement for t
    count[s.charCodeAt(i) - 97]++; // 97 = char code of 'a' – maps a→0, b→1, ...
    count[t.charCodeAt(i) - 97]--; // Subtract t's letters – anagram means all zeros
  }
  return count.every(c => c === 0); // true only if every letter count balanced
}

console.log(isAnagram("anagram", "nagaram")); // true
console.log(isAnagram("rat", "car")); // false
```

Line-by-line explanation (javascript)

Line	What it does
<code>function isAnagram(s, t) {</code>	Return true if t is rearrangement of s
<code>if (s.length !== t.length) return false;</code>	Different lengths cannot be anagrams
<code>const count = new Array(26).fill(0);</code>	Frequency bucket for a-z (26 letters)
<code>for (let i = 0; i < s.length; i++) {</code>	Single pass – increment for s, decrement for t
<code>count[s.charCodeAt(i) - 97]++;</code>	97 = char code of 'a' – maps a→0, b→1, ...
<code>count[t.charCodeAt(i) - 97]--;</code>	Subtract t's letters – anagram means all zeros
<code>return count.every(c => c === 0);</code>	true only if every letter count balanced
<code>console.log(isAnagram("anagram", "nagaram"));</code>	true
<code>console.log(isAnagram("rat", "car"));</code>	false

Expected output / acceptance criteria

- `isAnagram("anagram", "nagaram") → true`
- `isAnagram("rat", "car") → false`

Where to practice

- <https://leetcode.com/problems/valid-anagram/>

Common mistakes

- Sorting without noting $O(n \log n)$ in interview.
- Forgetting non-letter characters in general anagram problems.

80 LPA Interview Twist

Follow-up at Meta: Group Anagrams (LeetCode problem 49) with streaming input or Unicode — they want you to discuss hash key design (sorted string vs 26-character signature vs prime-product) and collision safety.

Interview connection

Grouping anagrams (LeetCode problem 49) and ransom note variants build on this.

Time estimate: 30 minutes

Exercise 6.3: LeetCode problem 121 – Best Time to Buy and Sell Stock

Problem statement

One transaction: find max profit from buying before selling given daily prices.

Learning objective

Single-pass track minimum price seen so far and max profit.

Prerequisites

- Arrays: [01-javascript-complete.md §5](#)

Step-by-step approach

1. Initialize `minPrice = Infinity`, `maxProfit = 0`.
2. For each `price`, update `maxProfit = max(maxProfit, price - minPrice)`.
3. Update `minPrice = min(minPrice, price)`.
4. Return `maxProfit`; handle decreasing-only array $\rightarrow 0$.
5. Avoid nested loops $O(n^2)$.

Hints

1. You must buy before you sell — track min **before** computing profit at each day.

2. `maxProfit` stays 0 if prices only fall.
3. This is "Kadane variant" on price differences.

Exact Answer

- `[7,1,5,3,6,4]` → 5 (buy at 1, sell at 6)
- `[7,6,4,3,1]` → 0
- Time: **linear** (one pass), Space: **constant** (only two variables)

Complete Solution

```
function maxProfit(prices) {           // One transaction max – buy once, sell once
  let minPrice = Infinity;             // Cheapest price seen so far (start high)
  let maxProfit = 0;                   // Best profit found so far

  for (const price of prices) {        // Walk day by day left to right
    minPrice = Math.min(minPrice, price); // Update cheapest buy price
    maxProfit = Math.max(maxProfit, price - minPrice); // Best sell today minus cheapest buy before today
  }
  return maxProfit;                    // 0 if prices only decrease
}

console.log(maxProfit([7, 1, 5, 3, 6, 4])); // 5 – buy at 1, sell at 6
console.log(maxProfit([7, 6, 4, 3, 1]));    // 0 – no profitable trade
```

Line-by-line explanation (javascript)

Line	What it does
<code>function maxProfit(prices) {</code>	One transaction max – buy once, sell once
<code>let minPrice = Infinity;</code>	Cheapest price seen so far (start high)
<code>let maxProfit = 0;</code>	Best profit found so far
<code>for (const price of prices) {</code>	Walk day by day left to right
<code>minPrice = Math.min(minPrice, price);</code>	Update cheapest buy price
<code>maxProfit = Math.max(maxProfit, price - minPrice);</code>	Best sell today minus cheapest buy before today
<code>return maxProfit;</code>	0 if prices only decrease
<code>console.log(maxProfit([7, 1, 5, 3, 6, 4]));</code>	5 – buy at 1, sell at 6
<code>console.log(maxProfit([7, 6, 4, 3, 1]));</code>	0 – no profitable trade

Expected output / acceptance criteria

- `[7,1,5,3,6,4]` → 5 (buy 1, sell 6)
- `[7,6,4,3,1]` → 0

Where to practice

- <https://leetcode.com/problems/best-time-to-buy-and-sell-stock/>

Common mistakes

- Finding global min and max without order constraint.
- Using two loops when one pass suffices.

80 LPA Interview Twist

Google extends to unlimited transactions (LeetCode problem 122), cooldown (LeetCode problem 309), and transaction fee (LeetCode problem 714) — expect state-machine dynamic programming formulation, not just the one-pass greedy.

Interview connection

Leads to LeetCode problems 122, 123, 188 (multiple transactions) — state machine dynamic programming family.

Time estimate: 35 minutes

Day 7 – Array Methods Review & LeetCode problems 217, 53

Exercise 7.1: LeetCode problem 217 – Contains Duplicate

Problem statement

Return true if any value appears at least twice in `nums`.

Learning objective

Use `Set` (collection of unique values) for **linear time** duplicate detection.

Step-by-step approach

1. Iterate `nums`; if `set.has(n)` return true; else `set.add(n)`.
2. Alternative: `nums.length !== new Set(nums).size`.
3. Compare with sorting adjacent duplicates (**$n \log n$ time**).
4. Submit and document complexity.

Hints

1. Set size vs array length is one-liner but explain loop version too.
2. Empty array → false.
3. Single element → false.

Exact Answer

- `[1,2,3,1] → true ; [1,2,3,4] → false`
- Time: **linear**, Space: **linear** with Set

Complete Solution

```
function containsDuplicate(nums) { // Return true if any value appears twice
  const seen = new Set();         // Set gives constant-time lookup
  for (const n of nums) {         // Check each number once (linear time)
    if (seen.has(n)) return true; // Already in set – duplicate found
    seen.add(n);                  // First time seeing n – remember it
  }
  return false;                   // No duplicates
}

console.log(containsDuplicate([1, 2, 3, 1])); // true – 1 appears twice
console.log(containsDuplicate([1, 2, 3, 4])); // false – all unique
```

Line-by-line explanation (javascript)

Line	What it does
<code>function containsDuplicate(nums) {</code>	Return true if any value appears twice
<code>const seen = new Set();</code>	Set gives constant-time lookup
<code>for (const n of nums) {</code>	Check each number once (linear time)
<code>if (seen.has(n)) return true;</code>	Already in set – duplicate found
<code>seen.add(n);</code>	First time seeing n – remember it
<code>return false;</code>	No duplicates
<code>console.log(containsDuplicate([1, 2, 3, 1]));</code>	true – 1 appears twice
<code>console.log(containsDuplicate([1, 2, 3, 4]));</code>	false – all unique

Expected output / acceptance criteria

- `[1,2,3,1] → true; [1,2,3,4] → false`

Where to practice

- <https://leetcode.com/problems/contains-duplicate/>

Common mistakes

- Using nested loops in interview without mentioning optimization path.

80 LPA Interview Twist

Follow-up LeetCode problem 219 (duplicate within distance k) requires sliding window + Set — Meta asks you to compare linear space Set vs window-sized space and justify for streaming data.

Interview connection

LeetCode problem 219 (duplicate within k distance) adds sliding window.

Time estimate: 20 minutes

Exercise 7.2: LeetCode problem 53 – Maximum Subarray

Problem statement

Find contiguous subarray with largest sum; return the sum.

Learning objective

Apply **Kadane's algorithm** for **linear time** maximum subarray.

Step-by-step approach

1. Track `currentSum` and `maxSum`.
2. For each num: `currentSum = max(num, currentSum + num)`.
3. Update `maxSum = max(maxSum, currentSum)`.
4. Walk through example `[-2, 1, -3, 4, -1, 2, 1, -5, 4]` on paper.
5. Explain why greedy extension works.

Hints

1. Reset `currentSum` when adding previous segment hurts.
2. All negative array → answer is max single element.
3. Don't confuse with max **product** (LeetCode problem 152).

Exact Answer

- `[-2, 1, -3, 4, -1, 2, 1, -5, 4] → 6` (subarray `[4, -1, 2, 1]`)
- Time: **linear** (one pass), Space: **constant** (Kadane's algorithm)

Complete Solution

```
function maxSubArray(nums) { // Kadane's algorithm – max sum contiguous
  let currentSum = nums[0]; // Best sum ending at current position
  let maxSum = nums[0]; // Global best sum seen

  for (let i = 1; i < nums.length; i++) { // Start from index 1 (0 already in curr
    currentSum = Math.max(nums[i], currentSum + nums[i]); // Extend subarray or st
    maxSum = Math.max(maxSum, currentSum); // Update global max
  }
  return maxSum;
}

console.log(maxSubArray([-2, 1, -3, 4, -1, 2, 1, -5, 4])); // 6 – subarray [4,-1,2,1]
```

Line-by-line explanation (javascript)

Line	What it does
<code>function maxSubArray(nums) {</code>	Kadane's algorithm – max sum contiguous subarray
<code>let currentSum = nums[0];</code>	Best sum ending at current position
<code>let maxSum = nums[0];</code>	Global best sum seen
<code>for (let i = 1; i < nums.length; i++) {</code>	Start from index 1 (0 already in currentSum)
<code>currentSum = Math.max(nums[i], currentSum + nums[i]);</code>	Extend subarray or start fresh at nums[i]
<code>maxSum = Math.max(maxSum, currentSum);</code>	Update global max
<code>console.log(maxSubArray([-2, 1, -3, 4, -1, 2, 1, -5, 4]));</code>	6 – subarray [4,-1,2,1]

Expected output / acceptance criteria

- `[-2,1,-3,4,-1,2,1,-5,4] → 6`

Where to practice

- <https://leetcode.com/problems/maximum-subarray/>

Common mistakes

- Starting `maxSum` at 0 when all negatives exist.
- Returning subarray indices when only sum requested.

80 LPA Interview Twist

Amazon/Google ask for divide-and-conquer $n \log n$ proof sketch, or return the actual subarray indices — plus variant LeetCode problem 918 (circular max subarray) requiring two Kadane passes.

Interview connection

Classic dynamic programming intro; divide-and-conquer $n \log n$ is follow-up for advanced rounds.

Time estimate: 40 minutes

Exercise 7.3: Week 1 Review – Rewrite From Memory

Problem statement

Without looking at solutions, re-implement: FizzBuzz, closure counter, array dedupe/rotate, and LeetCode problems 1, 242, 121, 217, 53 in one session.

Learning objective

Move patterns from short-term memory to fluency.

Prerequisites

- All Week 1 theory sections

Step-by-step approach

1. Set 3-hour timer; no IDE autocomplete hints from old files.
2. Order: warm-up FizzBuzz → counter → arrays → 5 LeetCode problems timed 25 min each max.
3. Mark problems needing re-study.
4. Push final code to `mern-mastery/week-01/review`.
5. Create GitHub repo if not done: <https://github.com/new>

Hints

1. If stuck 10 min, peek at hint tier 1 only, then retry.
2. Say complexity aloud after each solution.
3. Compare with week-01 solutions only after full pass.

Exact Answer

- Target: 5/7 exercises completed solo in one 3-hour session
- Each LeetCode solution should be recallable in ≤ 15 minutes with stated **linear time** complexity
- GitHub repo `mern-mastery/week-01/` contains all implementations

Complete Solution

```
// Review session checklist – re-implement all from memory, then compare with solu

// 1. FizzBuzz (getFizzBuzzLabel + fizzBuzz loop) – tests modulo and loop order
// 2. createCounter with increment/decrement/reset/getValue – tests closures
// 3. dedupe, secondLargest, rotateRight – tests array manipulation
// 4. twoSum – Map, linear time – tests hash map pattern
// 5. isAnagram – frequency array, linear time – tests character counting
// 6. maxProfit – single pass, linear time – tests greedy tracking
// 7. containsDuplicate – Set, linear time – tests Set lookup
// 8. maxSubArray – Kadane, linear time – tests dynamic programming intro

// Paste your implementations in week-01/review/review.js and run:
// node week-01/review/review.js – verify all outputs match Expected output section
```

Expected output / acceptance criteria

- At least 5/7 exercises completed solo.
- GitHub repo `mern-mastery` contains `week-01/` folder.

Where to practice

- GitHub: <https://github.com/new>
- LeetCode list: <https://leetcode.com/list/>

Common mistakes

- Skipping review day and forgetting closure patterns by Week 3.

80 LPA Interview Twist

At Google/Meta on-site, expect a 45-minute "implement 3 problems from scratch" round with no IDE hints — spaced repetition this week is the difference between fluency and "I solved it once on LeetCode."

Interview connection

Spaced repetition this week prevents "I solved it once" syndrome before phone screens.

Time estimate: 3 hours

Week 1 Completion Checklist

Item	Done
Day 1 variable quiz explained	☐
FizzBuzz implemented	☐
Closure counter independent instances	☐
<code>this</code> quiz mastered	☐
LeetCode problems 1, 242, 121, 217, 53 accepted	☐
Array manipulation trio complete	☐
Code pushed to GitHub <code>week-01/</code>	☐

Total estimated time: ~18–22 hours (matches roadmap Mon–Sun)